



Materi Golang Backend 3

Bahas **array**, **method**, dan **error handling** di Go backend dengan penjelasan sederhana dan dua latihan penguatan.



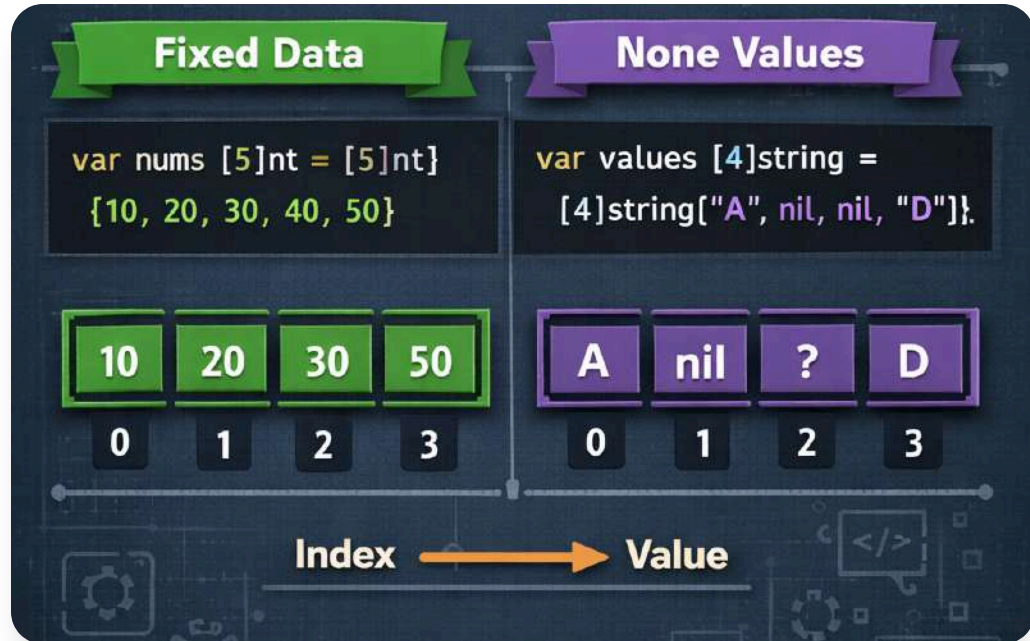
Juara Coding

June 2026



Tujuan Pembelajaran

Peserta memahami penggunaan array, pembuatan method pada struct, dan penanganan error untuk membangun backend yang rapi, aman, dan mudah dirawat.



Pengenalan Array di Go

Array menyimpan elemen bertipe sama dengan jumlah tetap. Di Go, panjang array adalah bagian dari tipe dan cocok saat data sudah diketahui sejak awal.

Cara Membuat Array

```
// JavaScript let numbers = [1, 2, 3, 4, 5];  
  
// Java int[] numbers = {1, 2, 3, 4, 5};  
  
// C++ int numbers[] = {1, 2, 3, 4, 5};  
  
# Ruby numbers = [1, 2, 3, 4, 5]
```

01 Format Penulisan

Array ditulis dengan format *[jumlah]tipeData*, misalnya *[3]int* untuk tiga elemen bertipe int.

03 Nilai Default

Jika tanpa nilai awal, elemen array berisi nilai default, seperti *0* untuk int dan string kosong untuk string.

02 Contoh Deklarasi

Penulisan *[3]int* berarti array berisi tiga angka dengan tipe data *int*.

Contoh Array Sederhana

```
// Array Sederhana Example  
  
numbers := [5]int{10, 20, 30, 40, 50}  
  
first := numbers[0]           // Access first element  
third := numbers[2]           // Access third element  
slice := numbers[1:4]          // Slice of elements  
  
fmt.Println("First:", first)  
fmt.Println("Third:", third)  
fmt.Println("Slice:", slice)  
  
}
```

Contoh ini menunjukkan pengisian array berdasarkan indeks, dengan indeks dimulai dari 0 hingga posisi terakhir.



Deklarasi Array

Array angka dibuat dengan panjang 3 dan tipe data integer.



Isi per Indeks

Nilai diisi ke posisi 0, 1, dan 2 menggunakan akses indeks secara langsung.



Indeks Dimulai dari 0

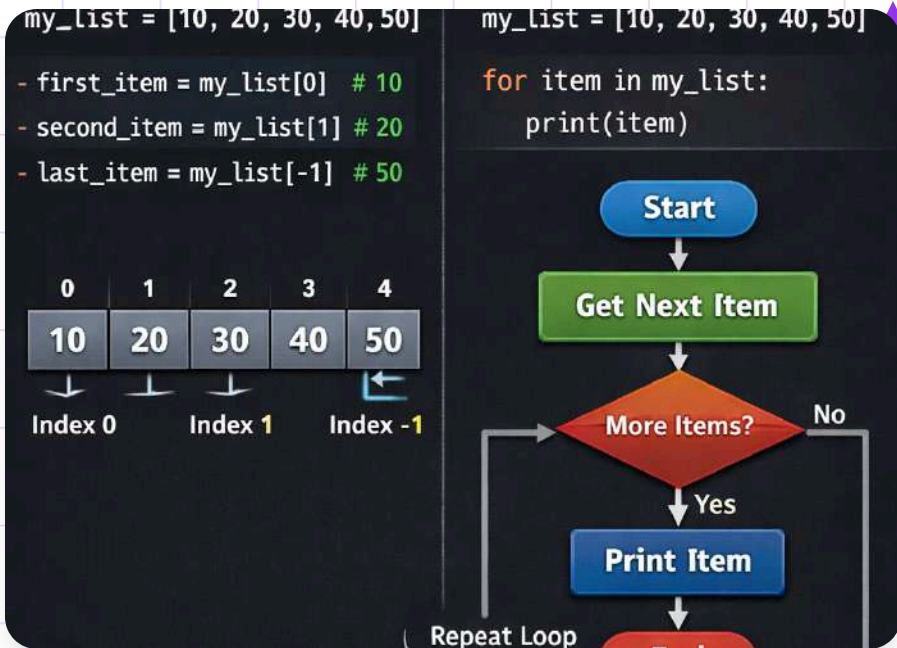
Elemen pertama berada di indeks 0, sedangkan elemen terakhir di panjang array dikurangi 1.

```
days := ([]*string{
    "Monday", "Tuesday", ..., "Wednesday", None, None,
    "Thursday", None, None, None;
});
```

Inisialisasi Array

Array dapat langsung diisi saat dibuat, misalnya dengan nilai hari. Cara ini ringkas dan urutan elemen mengikuti penulisan nilainya.

Mengakses dan Menampilkan Array



Elemen array diakses dengan indeks seperti `data[1]`, lalu dapat ditampilkan langsung atau satu per satu memakai perulangan.



Akses dengan Indeks

Setiap elemen array dapat diambil menggunakan indeks, misalnya `data[1]`.



Tampilkan Langsung

Isi array bisa dicetak langsung melalui variabel array untuk melihat seluruh data.



Gunakan Perulangan

Perulangan membantu menampilkan elemen satu per satu dengan lebih efisien dan rapi.

Perulangan pada Array

```
import "fmt"

func main() {

    numbers := []int{1, 2, 3, 4, 5}

    for i, num := range numbers {
        fmt.Printf("Index: %d, Value: %d\n", i, num)
    }
}
```

01 Sintaks range

Gunakan *range* untuk mengambil **indeks** dan **nilai** array dalam satu perulangan.

02 Contoh hasil iterasi

Pada array [10, 20, 30], variabel *i* berisi posisi elemen dan *v* berisi nilainya.

03 Manfaat di backend

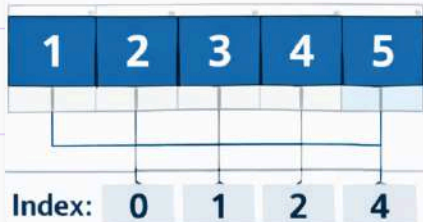
Pola ini memudahkan pemrosesan daftar data secara berurutan, rapi, dan mudah dibaca.

Array dan Slice

Array

- Fixed Size
- Cannot be Resized
- Full Underlying Data

```
arr := [5]int{1, 2, 3, 4, 5}
```



Slice

- Dynamic Size
- Can be Resized
- View of Underlying Array

```
s := arr[1:4]
```



Array berukuran tetap, sedangkan *slice* lebih fleksibel dan lebih sering dipakai dalam backend Go.



Ukuran Tetap

Array memiliki panjang tetap sejak dibuat dan tidak dapat berubah saat program berjalan.



Lebih Fleksibel

Slice dapat bertambah atau berkurang ukurannya sesuai kebutuhan pengolahan data.



Penting Dipahami

Memahami *array* penting karena *slice* dibangun dari konsep struktur data yang serupa.

```
struct Person {  
    name: String,  
    age: u32,  
    city: String,  
}
```

Pengenalan Method

Method adalah fungsi pada tipe tertentu, biasanya **struct**, agar data dan perilaku terkait tersusun lebih rapi.

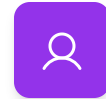
Struct dan Method

```
    Radius float64
}

func (c *Circle) Area() float64 {
    return 3.14 * c.Radius * c.Radius
}

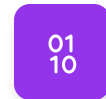
func main() {
    c := Circle(Radius: 5.0)
    area := c.Area()
    fmt.Println("Area:", area)
}
```

Contoh ini menunjukkan bahwa *SayHello* adalah method milik struct *User* dengan receiver (*u User*).



Struct User

Struct *User* menyimpan data *Name* yang digunakan oleh method.



Method SayHello

Method *SayHello()* mencetak sapaan memakai nilai *Name* dari instance *User*.



Receiver

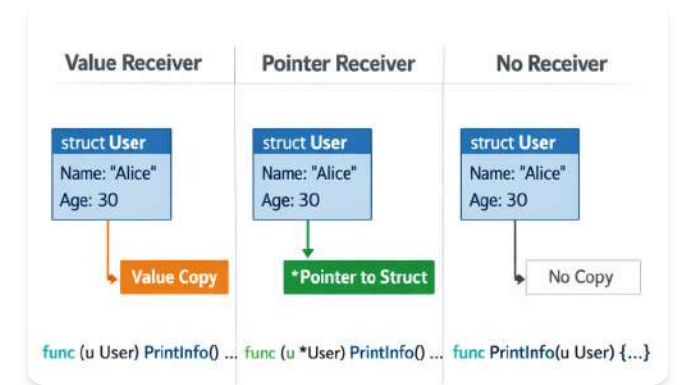
Receiver (*u User*) menandakan method bekerja pada data bertipe *User*.

```
type Rectangle struct {  
    Width : float64  
    Height float64  
}  
  
func (r Rectangle) Area() float64 {  
    return r.Width * r.Height  
}  
  
func main() {  
  
    rect = Rectangle(Width: 5.0, Height: 3.0)  
    area = rect.Area()  
}
```

Cara Memanggil Method

Method dipanggil lewat titik pada objek. Umum untuk validasi dan formatting struct.

Value Receiver dan Pointer Receiver



01 Value receiver

Menerima salinan data, sehingga perubahan di dalam method tidak mengubah nilai asli.

03 Kapan digunakan

Pointer receiver umum dipakai saat struct besar atau ketika isi data memang perlu diubah.

02 Pointer receiver

Menerima alamat data, sehingga perubahan di dalam method akan memengaruhi data asli.

Contoh Pointer Receiver



Method ini mengubah nilai Name pada objek asli dan penting untuk pembaruan data struct di backend.



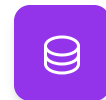
Ubah objek asli

Pointer receiver memungkinkan method memperbarui field langsung pada struct yang sama.



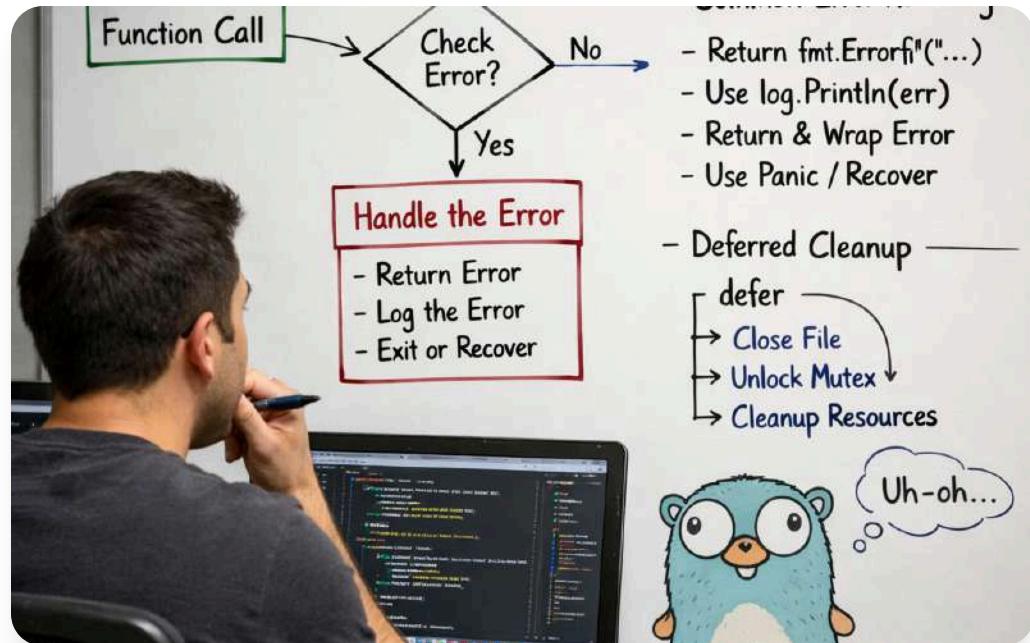
Penting di backend

Berguna saat mengubah status user, profil, atau hasil pemrosesan request.



Efisien untuk struct

Menghindari penyalinan data saat method bekerja pada struct yang lebih besar.



Pengenalan Error Handling

Dalam Go, *error handling* dilakukan lewat nilai error yang dikembalikan fungsi, sehingga alur kegagalan lebih aman, jelas, dan mudah dilacak.

Bentuk Dasar Error di Go

```
import (
    "fmt"
    "errors"
)

func main() {
    result, err := fetchData()
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println("Result", result)
    value := getValue()
    if value == nil {
        fmt.Println("Value is None")
    } else {
        fmt.Println("Value", value)
    }
}

// Output:
Error: failed to fetch data
```

Pola *if err != nil* adalah cara umum di Go untuk memeriksa error dan menangani masalah secepat mungkin.



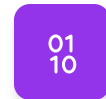
Cek Setelah Operasi

Setelah fungsi dijalankan, nilai *err* diperiksa untuk memastikan proses berhasil.



Tangani Lebih Awal

Jika ada masalah, program bisa langsung mencetak pesan, berhenti, atau mengembalikan nilai.



Pola Umum di Go

Struktur *if err != nil* sering dipakai karena sederhana, jelas, dan mudah dibaca.

```

4
5 func fetchData(id int)(string, error) {
6     if id == 0
7         return "", errors.New("invalid ID ")
8     return "Data for ID: " + fmt.Sprintf(id), None
9 }
10
11 func main() {
12     result, err := fetchData(0)
13     if err != None {
14         handleErr(err)
15     } return
16     fmt.Println("Result:", result)
17 }
18
19 func handleErr(err error)
20     fmt.Println("Error", err.Error())
21 }
22
23 func None() error

```

Membuat Error Sendiri

Buat error kustom agar pesan lebih jelas dan sesuai dengan logika program saat validasi gagal.

Pentingnya Error Handling



Mencegah crash sistem

Menangani kegagalan agar aplikasi tidak berhenti mendadak saat terjadi masalah di backend.



Sistem lebih stabil

Penanganan error yang baik memudahkan perbaikan dan meningkatkan keandalan layanan backend.



Respons lebih aman

Error handling membantu memberi respons yang aman saat input tidak valid, database gagal, atau file hilang.



Latihan Soal 1

```
import "fmt"

func main() {
    // Define an array of numbers
    numbers := []int{3, 7, 2, 8, 4}

    sum := 0

    // Loop through the array
    for i := 0; i < len(numbers); i++ {
        sum += numbers[i]
    }

    // Print the result
    fmt.Println("The sum of the numbers is:", sum)
}
```

01 Simpan 5 Nilai Integer

Buat array berisi 5 bilangan bulat dengan nama variabel yang sederhana dan mudah dipahami.

03 Hitung Jumlah Elemen

Jumlahkan seluruh elemen array, lalu tampilkan hasil akhirnya setelah proses perulangan selesai.

02 Tampilkan dengan Range

Gunakan perulangan range untuk menampilkan semua nilai yang ada di dalam array secara berurutan.

Latihan Soal 2



01 Struct Product

Buat struct Product dengan field Name dan Price untuk menyimpan data produk.

03 Uji Dua Data

Jalankan satu contoh data valid dan satu contoh data dengan Price negatif untuk memicu error.

02 Method dan Validasi

Tambahkan method untuk menampilkan info produk dan fungsi error jika Price kurang dari 0.